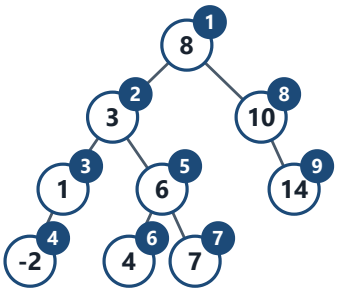
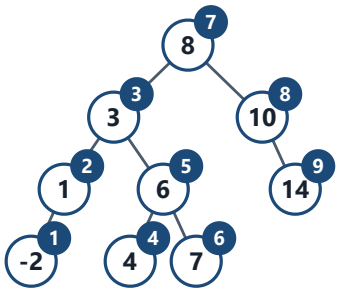
Cheatsheet DSA – tra nhanh

Sắp xếp · Tìm kiếm · Cây nhị phân · theo cách làm của cô Hồng. Đề: 3 ý a) duyệt cây, b) khai báo, c) viết hàm.

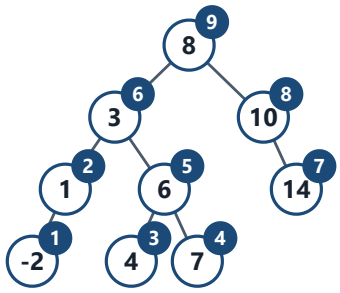
① Đề mẫu Câu 8: đáp án trọn bộ



Trước: 8 3 1 -2 6 4 7 10 14



Giữa: -2 1 3 4 6 7 8 10 14



Sau: -2 1 4 7 6 3 14 10 8

a) Trước (gốc-trái-phải): `8 3 1 -2 6 4 7 10 14` · Giữa (trái-gốc-phải): `-2 1 3 4 6 7 8 10 14` · Sau (trái-phải-gốc): `-2 1 4 7 6 3 14 10 8`

Số trong chấm xanh = thứ tự được thăm. Gốc 8 luôn đầu tiên ở duyệt trước, cuối cùng ở duyệt sau. Cây này là BST nên duyệt giữa tăng dần.

b) Khai báo

```
struct nutCay
{
    int duLieu;
    nutCay *trai;
    nutCay *phai;
};
```

c) Tổng nút dương (= 53)

```
long long tongDuong(nutCay *t)
{
    if (t == NULL) return 0;
    if (t->duLieu > 0)
        return t->duLieu
             + tongDuong(t->trai)
             + tongDuong(t->phai);
    return tongDuong(t->trai)
         + tongDuong(t->phai);
}
```

② Hàm đệ quy trên cây: một khung cho mọi bài

```
int f(nutCay *t)
{
    if (t == NULL) return 0;                // 1. cây rỗng: điều kiện dừng
    return PHẦN_NÚT + f(t->trai) + f(t->phai);   // 2. phần của nút  3. gộp trái + phải
}
```

Đề bài	PHẦN_NÚT	Đề bài	PHẦN_NÚT
Đếm nút	`1`	Tổng giá trị	`t->duLieu`
Đếm nút lẻ	`(t->duLieu % 2 != 0)`	Tổng nút dương	`(t->duLieu > 0 ? t->duLieu : 0)`
Đếm nút chẵn	`(t->duLieu % 2 == 0)`	Đếm nút dương	`(t->duLieu > 0)`
Đếm chia hết 3	`(t->duLieu % 3 == 0)`	Đếm nút 2 con	`(t->trai && t->phai)`

Đếm lá: nếu cả 2 con NULL trả 1, còn lại `demLa(trai) + demLa(phai)`.
Chiều cao: `1 + max(chieuCao(trai), chieuCao(phai))` ; rỗng = 0, chỉ gốc = 1.

Cây Câu 8: 9 nút · 4 lá (−2, 4, 7, 14) · cao 4 · 3 nút lẻ (3, 1, 7) · tổng 51.

Hay mất điểm: viết `(t->duLieu > 0)` mà quên nhân/cộng giá trị thì chỉ *đếm* chứ không *tổng*. Lẻ phải viết `!= 0`, không viết `== 1`.

Duyệt cây: ba hàm chỉ khác vị trí dòng `cout`

```
void duyetTruoc(nutCay *t) { if (t) { cout << t->duLieu << " ";  duyetTruoc(t->trai);  duyetTruoc(t->phai); } }
void duyetGiua (nutCay *t) { if (t) { duyetGiua(t->trai);  cout << t->duLieu << " ";  duyetGiua(t->phai); } }
void duyetSau  (nutCay *t) { if (t) { duyetSau(t->trai);  duyetSau(t->phai);  cout << t->duLieu << " "; } }
```

⑥ Lỗi hay mất điểm, kiểm trước khi nộp

- Duyệt cây: đọc đúng thứ tự (trước = gốc đầu, sau = gốc cuối), thăm hết **mọi** nút, nhớ cây có nút âm (−2).
- Khai báo: `struct nutCay` đủ 3 trường, kết thúc bằng `;` ; con trỏ `trai`, `phai` kiểu `nutCay *`.
- Hàm đệ quy: **có điều kiện dừng** (`t == NULL`), hai lần gọi trái và phải, trả đúng kiểu.
- Tổng nút dương: cộng `t->duLieu`, đừng chỉ đếm. Nút lẻ: `% 2 != 0`.
- BST: bổ sung dùng `nutCay *&t` (tham chiếu), không dùng `nutCay *t`.
- Hủy nút hai con: chép khóa thế mạng lên, hủy nút thế mạng cũ. Nói rõ dùng cách một hay cách hai.
- Tìm nhị phân: nhắc mảng phải tăng dần. Sắp giảm dần: đảo dấu so sánh, không đảo cả vòng lặp.
- Vẽ cây: nhị phân tìm kiếm không có khóa trùng; mỗi bước dựng cây ghi rõ khóa đang thêm.

③ Cây nhị phân tìm kiếm (BST)

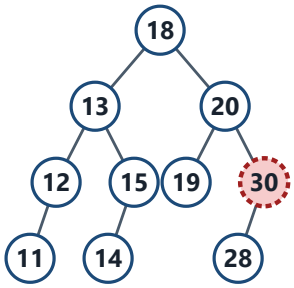
Quy tắc: trái < gốc < phải (mọi nút). Dựng: thêm lần lượt, nhỏ đi trái, lớn đi phải, gặp chỗ trống thì đặt. Trùng thì bỏ qua.

```
void boSung(nutCay *&t, int x)
{
    if (t == NULL) t = taoNut(x);
    else if (x < t->duLieu) boSung(t->trai, x);
    else if (x > t->duLieu) boSung(t->phai, x);
}
```

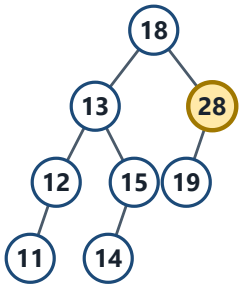
Hủy nút x (tìm x trước, rồi xét số con):

1. **Lá:** bỏ luôn (cha trỏ NULL).
2. **Một con:** cha trỏ thẳng tới con đó.
3. **Hai con:** chọn nút thế mạng, chép khóa của nó lên nút cần hủy, rồi hủy nút thế mạng cũ.
   - **Cách hai:** trái nhất của cây con phải.
   - **Cách một:** phải nhất của cây con trái.

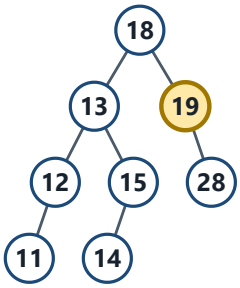
Tìm min: đi mãi sang trái. Tìm max: đi mãi sang phải. Duyệt giữa của BST luôn tăng dần.



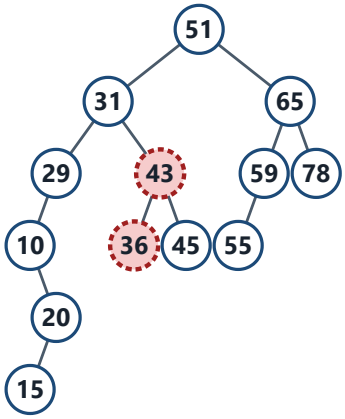
Bài 1a: dựng từ 18 13 15 12 20 11 30 19 28 14



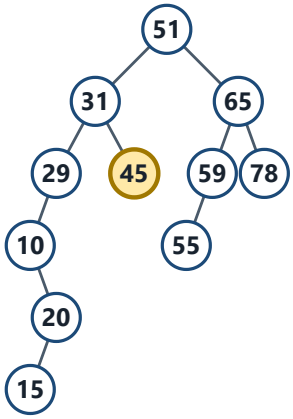
Sau hủy 30 rồi 20 (cách hai): 28 thế chỗ



Cùng bài, cách một: 19 thế chỗ



Bài 2a: dựng từ 51 31 43 29 65 10 20 36 78 59 15 45 55



Sau hủy 36 (lá) rồi 43 (một con): 45 nối lên

④ Sắp xếp: nhớ ý tưởng + sửa dấu để giảm dần

Phần tô vàng = chỗ cần đảo khi sắp **giảm dần** (code bên dưới là **tăng dần**; muốn **giảm dần** chỉ đảo các dấu tô vàng: < ↔ >, <= ↔ >=, nhỏ nhất ↔ lớn nhất).

Thuật toán	Ý tưởng một câu	Tốt nhất	Xấu nhất	Ổn định
Chọn	Mỗi lượt chọn phần tử nhỏ nhất đưa về đầu phần chưa sắp	n^2	n^2	không
Chèn	Lấy a[i] chèn vào đúng chỗ trong phần đầu đã sắp	n	n^2	có
Nổi bọt	So sánh cặp kề, đổi chỗ; phần tử lớn nhất nổi về cuối	n^2*	n^2	có
Trộn	Chia đôi, sắp từng nửa, trộn hai nửa đã sắp	n log n	n log n	có
Nhanh	Chọn pivot, chia ≤ pivot bên trái, ≥ pivot bên phải, sắp hai phía	n log n	n^2	không

* Bản có cờ dừng sớm (không đổi chỗ thì dừng) tốt nhất là n. Trộn cần thêm mảng phụ O(n).

④ (tiếp) Code năm thuật toán, sắp tăng dần

```
void SelectionSort(int a[], int n)
{
  for (int i = 0; i < n - 1; i++)
  {
    int vt = i;
    for (int j = i + 1; j < n; j++)
      if (a[j] < a[vt])
        vt = j;
    swap(a[i], a[vt]);
  }
}
```

```
void InsertionSort(int a[], int n)
{
  for (int i = 1; i < n; i++)
  {
    int x = a[i], j = i - 1;
    while (j >= 0 && a[j] > x)
    {
      a[j + 1] = a[j];
      j--;
    }
    a[j + 1] = x;
  }
}
```

```
void BubbleSort(int a[], int n)
{
  for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - 1 - i; j++)
      if (a[j] > a[j + 1])
        swap(a[j], a[j + 1]);
}
```

```
void QuickSort(int a[], int dau, int cuoi)
{
  int i = dau, j = cuoi, x = a[(dau + cuoi) / 2];
  while (i <= j)
  {
    while (a[i] < x) i++;
    while (a[j] > x) j--;
    if (i <= j)
    {
      swap(a[i], a[j]);
      i++;
      j--;
    }
  }
  if (dau < j) QuickSort(a, dau, j);
  if (i < cuoi) QuickSort(a, i, cuoi);
}
```

```
void Merge(int a[], int dau, int giua, int cuoi)
{
  int b[1000], i = dau, j = giua + 1, k = 0;
  while (i <= giua && j <= cuoi)
    if (a[i] <= a[j]) b[k++] = a[i++];
    else b[k++] = a[j++];
  while (i <= giua) b[k++] = a[i++];
  while (j <= cuoi) b[k++] = a[j++];
  for (i = dau, k = 0; i <= cuoi; i++) a[i] = b[k++];
}
void MergeSort(int a[], int dau, int cuoi)
{
  if (dau < cuoi)
  {
    int giua = (dau + cuoi) / 2;
    MergeSort(a, dau, giua);
    MergeSort(a, giua + 1, cuoi);
    Merge(a, dau, giua, cuoi);
  }
}
```

Ghi nhớ: Chọn: tìm min của phần còn lại rồi `swap` về vị trí `i` · Chèn: dồn phần tử lớn hơn `x` sang phải rồi đặt `x` · Nổi bọt: vòng trong chạy tới `n-1-i` · Nhanh: pivot là phần tử giữa, hai con trỏ `i`, `j` tiến vào nhau rồi gọi đệ quy hai phía · Trộn: `giua = (dau+cuoi)/2`, sắp hai nửa rồi `Merge`.

⑤ Tìm kiếm

```
int timTuanTu(int a[], int n, int x)
{
  for (int i = 0; i < n; i++)
    if (a[i] == x) return i;
  return -1;
}
int timNhiPhan(int a[], int n, int x)
{
  int dau = 0, cuoi = n - 1;
  while (dau <= cuoi)
  {
    int giua = (dau + cuoi) / 2;
    if (a[giua] == x) return giua;
    if (a[giua] < x) dau = giua + 1;
    else cuoi = giua - 1;
  }
  return -1;
}
```

Tuần tự: duyệt từ đầu, gặp x thì trả vị trí. O(n), mảng bất kỳ.
Nhị phân: mảng **đã tăng dần**. Lấy giữa, so với x, bỏ một nửa. O(log n). Quên sắp xếp trước thì kết quả sai.

Ví dụ 5 (cô): A = −8 −5 0 4 15 20 50 63 75 98, x = 20 (vị trí đếm từ 1)
Lượt 1: [1..10], giữa = vị trí 5 → 15 < 20 → [6..10] · Lượt 2: giữa = vị trí 8 → 63 > 20 → [6..7]
Lượt 3: giữa = vị trí 6 → 20 = x → **có trong mảng**. (code 0-based trả chỉ số 5)